

# Spring Security

19 September 2026 09:22

## Why does our backend need security ?

Ans: Suppose we build BookMyShow Application with these API's

POST : /api/user/register  
POST : /api/bookings  
GET : /api/users/7/booking  
POST : /api/admin/movies  
DELETE : /api/admin/show/15

Without security, anybody can:

1. See another user's booking
2. Create a booking using another user's id
3. Add or delete movies

Note :

1. spring security is very important for backend application
2. To protect our application and data we need to implement spring security logic
3. Spring security concept we can use to secure our web application / REST API's

Spring security help us Answer us two questions for every request.

1. Who is making this request ?
2. Is this person allowed to perform this action ?

## Authentication And Authorization

---

Authentication means verifying identity.

Questions: who are you ?

Answer : I am abhay@gmail.com, and here is my password for proof.

Examples:

Username and password

OTP

Fingerprint

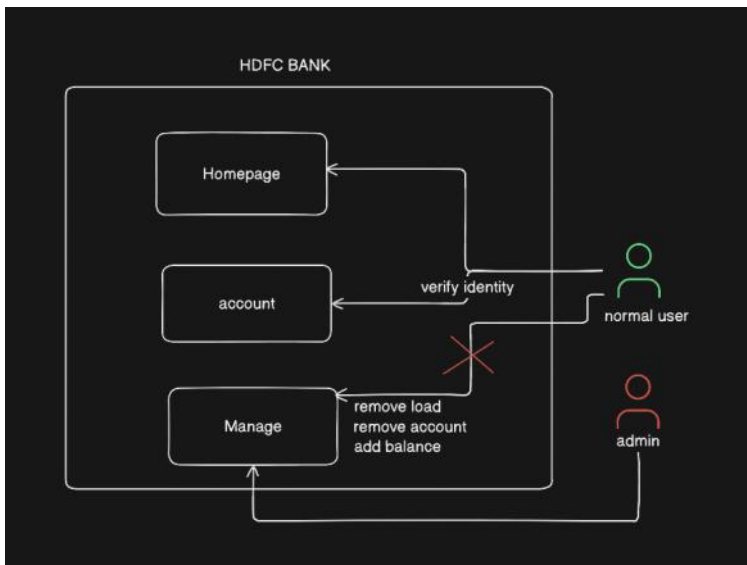
Google login

JWT bearer token

## Authorization

---

Means checking permission after identity is known.



Question: you are abhay, but are you allowed to delete movie ?  
 Ans : NO, abhay has USER role, only ADMIN can delete a movie.

## Difference

| Authentication            | Authorization                |
|---------------------------|------------------------------|
| Confirm identity          | Confirm permission           |
| Happen first              | Happens after authentication |
| Login/pass/token          | Role/authority/access rule   |
| Failure normally give 401 | Failure normally give 403    |

Note: to secure our spring boot application we need to add below starter in pom.xml file

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

## Limitation current approach:

The username and password are temporary , we cannot define our own users or decide which endpoint should be public.

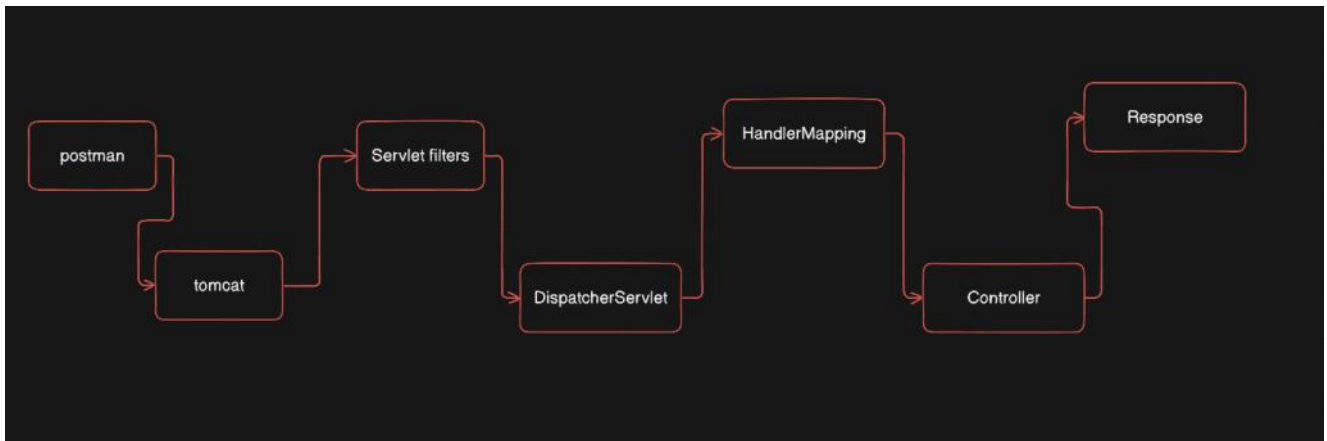
Can we change these username and password ?

## Spring Security is a Filter based System

Before spring security filters, first understand how a normal request reached a spring boot controller.

What happens when we call a Spring boot API ?  
API: /api/movies/10

Request flow

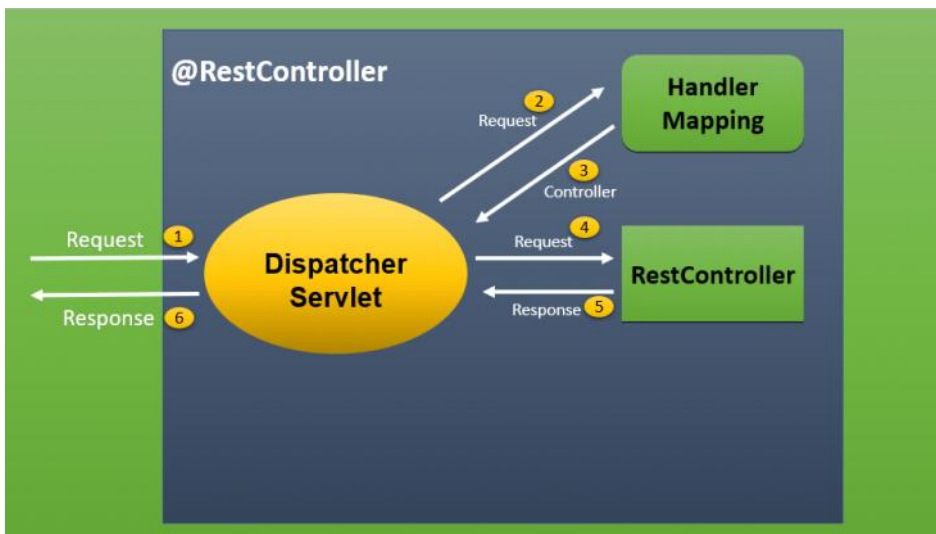


What is DispatcherServlet ?

DispatcherServlet is the front controller of spring.

Front controller means one central component receives Spring MVC request and sends each request to the correct controller method.

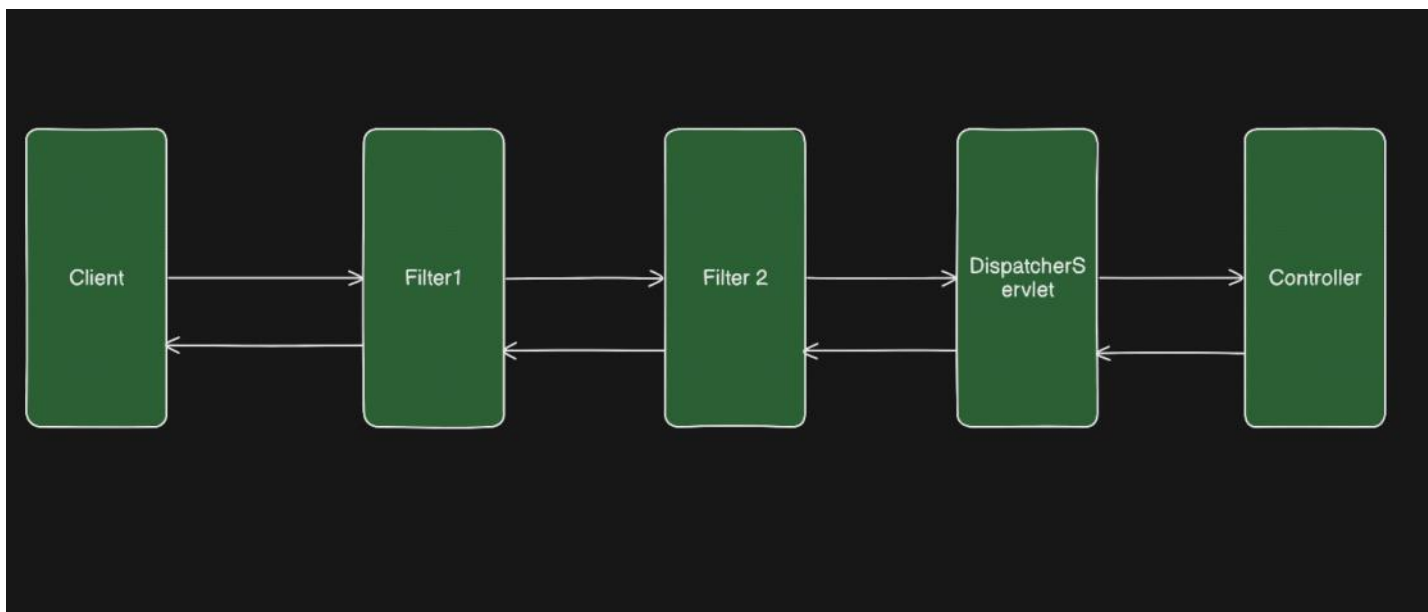
Note: This **DispatcherServlet** does not contain our business logic. It coordinates others with spring component



What is Filter ?

A filter is Servlet component that can inspect or modify HTTP request before it reaches a servlet and inspect or modify the response while it returns.

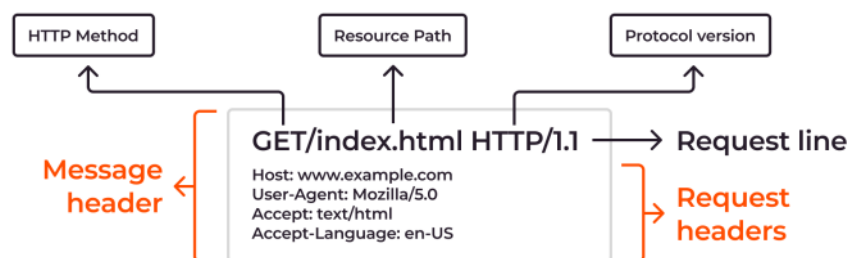




A filter can:

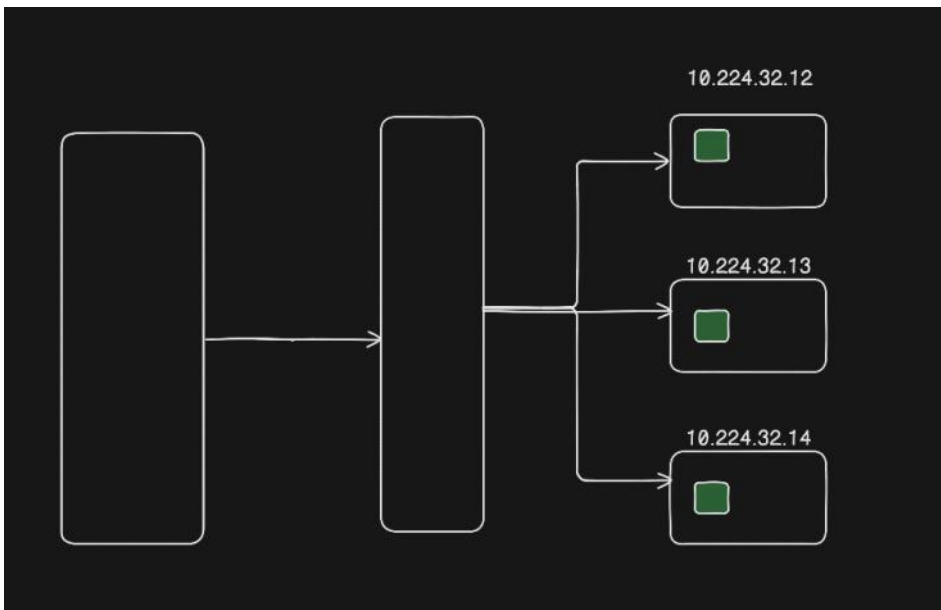
1. Allow the req to continue
2. Reject the request immediately
3. Add or read headers
4. Modify req
5. Perform work before and after the controller

### HTTP Request Example



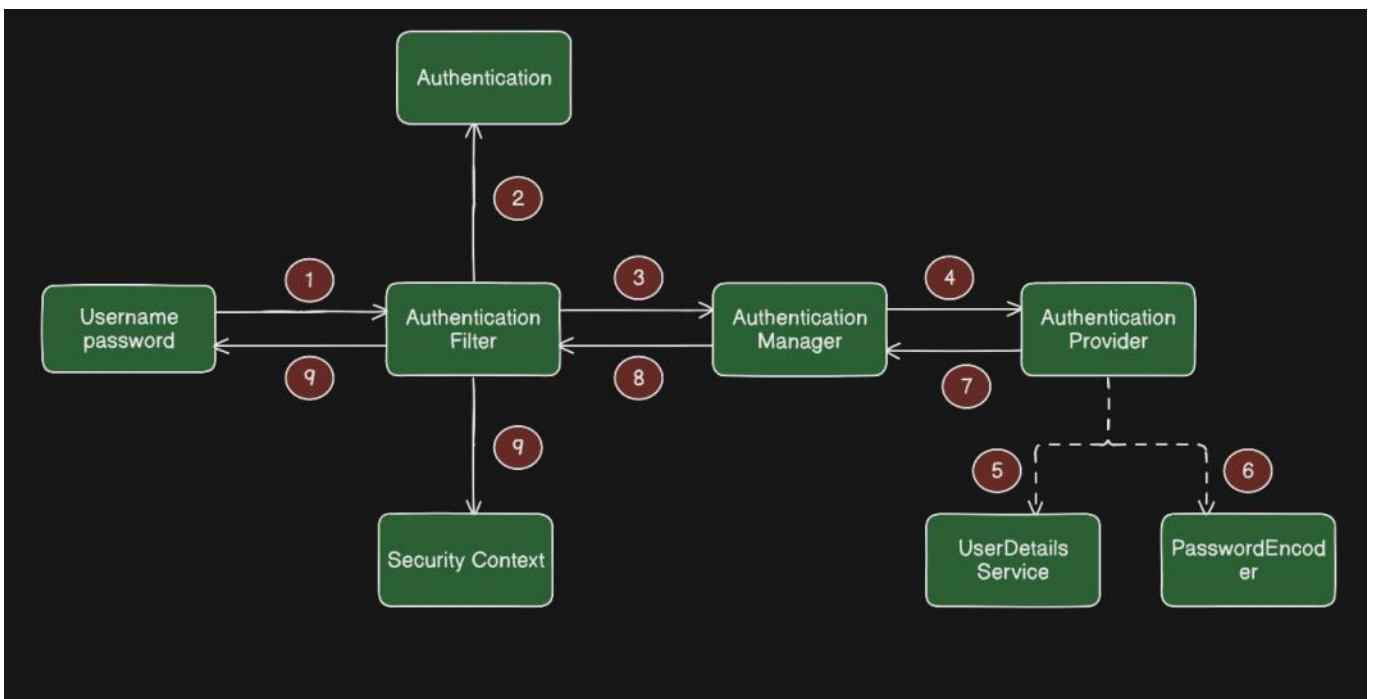
Common filter use case?

1. Authentication
2. Authorization
3. Request logging



First see the complete Authentication Internal Flow

---



Note: the password does not directly go to controller, spring security verifies identity before the controller is allowed to run.

AuthenticationProvider:

---

It knows how to verify one type of credentials

For database username/password login, spring normally uses DaoAuthenticationProvider.

UserDetailsService

---

It loads user by username, it does not compare passwords it only loads user data.

UserDetails:

---

It hold user data into the format spring security understand.

Username

Encoded password

Authorities

PasswordEncoder

It hashes password and verifies raw password again stored hash.

```
String hash = passwordEncoder.encode("root");
```

```
Boolean match = passwordEncoder.matches("root",hash);
```

AuthenticationManager:

It receives an **unauthenticated** Authentication object and ask for authentication to be performed.

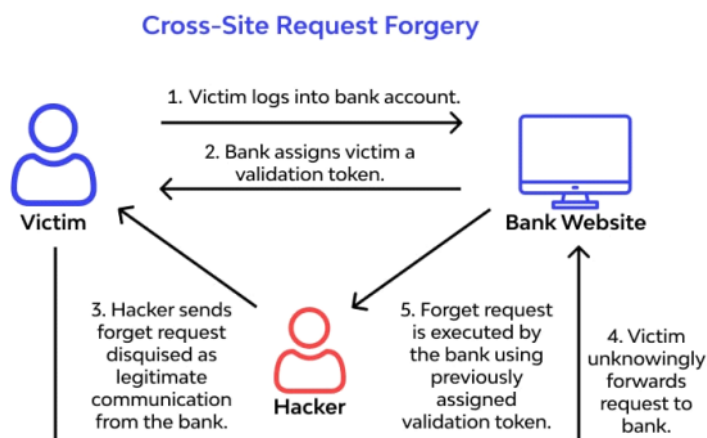
AuthenticationManager is an interface it s common implementation is ProviderManager

Authentication

It is interface represent an authentication request.

SecurityContextHolder

After successful authentication, spring store the **Authentication** in **SecurityContext**

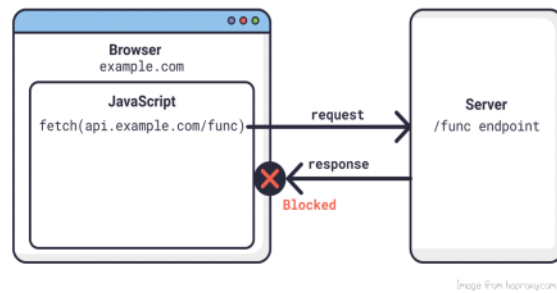


Note: Authentication will not do spring boot will do it automatically

CORS:

-----

# CORS Explained for Beginners



## Role & Authorities

-----

### Authority:

-----

An authority is a specific permission

MOVI\_READ  
MOVIE\_CREATE  
MOVIE\_DELETE

### Role

-----

A role is broader user category

USER  
ADMIN

In spring security `hasRole("ADMIN")` check for authority `ROLE_ADMIN`

Razorpay ==>

BMS + payment ==> razorpay ==> full backend

Razorpay ==>